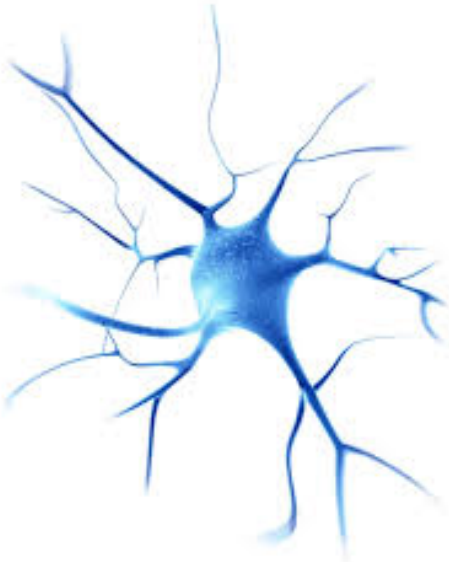
Reconnaissance Vocale

Classification de Commandes Vocales par Deep Learning



Réalisé par :
ZIYTER Fatima-Ezzahrae
El Allam Maha

Encadrante :
Mme. El Hlouli

Filière : GI-ILSI

Génie Informatique - Ingénierie Logicielle
et Systèmes Intelligents

Année Universitaire 2025 – 2026

Table des matières

Introduction Générale 2

Architecture Globale du Pipeline 3
1 Vision d'Ensemble du Système . . . 3
2 Choix Technologiques . . . 3
3 Architecture Logicielle . . . 4

Prétraitement des Données 5
4 Chaîne de Prétraitement et Extraction de Caractéristiques . . . 5

Conception du Modèle CNN 7
5 Justification du choix CNN 2D . . . 7
6 Architecture détaillée . . . 7
7 Fonction de perte . . . 8
8 Stratégie d'entraînement . . . 8

Résultats et Analyse CNN 9
9 Accuracy finale . . . 9
10 Matrice de confusion . . . 10
11 Analyse ROC . . . 11
12 Analyse de l'Overfitting et Underfitting . . . 12
13 Limites du modèle . . . 13
14 Comparaison avec autres approches . . . 13

Approche Séquentielle : Modèle LSTM 14
15 Conception de l'Architecture LSTM . . . 14
16 Résultats Expérimentaux . . . 14
17 Synthèse Comparative . . . 15

Architecture Backend et Inférence 16
18 Pipeline d'inférence . . . 16
19 Prédiction . . . 17
20 Format de sortie . . . 17
21 Technologies utilisées . . . 17
22 Avantages . . . 17

Démonstration et Inférence 18

Conclusion Générale 19

Introduction Générale

La reconnaissance automatique de la parole constitue aujourd'hui un axe central de l'intelligence artificielle appliquée. Elle vise à transformer un signal acoustique en information symbolique exploitable par une machine. Parmi ses sous-domaines, la reconnaissance de commandes vocales courtes représente un cas d'étude particulièrement pertinent : il s'agit de classifier des mots isolés tels que *yes*, *no*, *stop*, *go*, *left* ou *right*, à partir d'un enregistrement audio.

Ce problème s'inscrit dans un cadre de classification supervisée multi-classe sur données temporelles. Le principal défi réside dans la nature même du signal vocal : une onde continue échantillonnée à haute fréquence (16 kHz dans notre cas), présentant une forte redondance, une variabilité inter-locuteurs et une sensibilité aux conditions d'enregistrement. Sous sa forme brute, ce signal n'est pas directement adapté à l'apprentissage profond.

La question centrale de ce projet peut ainsi être formulée de manière synthétique :

> Comment transformer un signal audio brut en une représentation structurée, compacte et robuste, permettant à un modèle de Deep Learning de classifier efficacement des commandes vocales ?

Pour répondre à cette problématique, nous adoptons un pipeline complet comprenant :
- une étape de prétraitement et d'homogénéisation des données,
- l'extraction de caractéristiques perceptuelles (MFCC),
- la conception d'un réseau convolutionnel 2D,
- et une évaluation expérimentale rigoureuse.

L'objectif du projet est donc la conception et l'implémentation d'un système de reconnaissance de commandes vocales fondé sur une articulation cohérente entre traitement du signal et apprentissage profond. Au-delà de la performance du modèle, une attention particulière est portée à la structuration modulaire du projet, à la reproductibilité expérimentale et à la clarté architecturale du pipeline.

Architecture Globale du Pipeline

1 Vision d'Ensemble du Système

Le système développé repose sur une architecture modulaire transformant un signal de parole brut en une prédiction catégorielle exploitable. L'objectif est d'assurer une continuité logique entre l'acquisition du signal, la transformation spectrale et l'apprentissage supervisé via deux approches complémentaires : spatiale et temporelle.

La chaîne de traitement adoptée peut être synthétisée comme suit :

Audio → Prétraitement → MFCC → **Dual Modelling (CNN / LSTM)** → Prédiction

Cette architecture séquentielle organise la transformation progressive de l'information :

- **Signal audio :** entrée numérique échantillonnée à 16 kHz représentant l'évolution temporelle de la pression acoustique.
- **Prétraitement :** homogénéisation des données (rééchantillonnage, normalisation, ajustement dimensionnel) afin de garantir la cohérence statistique du dataset.
- **Extraction des MFCC :** transformation du signal en une représentation temps–fréquence compacte. Cette matrice sert d'entrée commune aux deux architectures de Deep Learning.
- **Réseau CNN 2D :** modèle convolutionnel chargé d'extraire automatiquement des motifs discriminants (signatures spectrales) dans la matrice MFCC, traitée ici comme une image.
- **Réseau LSTM :** modèle récurrent conçu pour traiter les MFCC comme une série temporelle, capturant ainsi la dynamique et les dépendances séquentielles de la parole.
- **Prédiction :** estimation probabiliste des classes via une couche de sortie Softmax, permettant de classifier la commande vocale.

Cette structuration permet une séparation claire entre le traitement du signal et la phase d'apprentissage, améliorant la lisibilité, la maintenabilité et l'évolutivité du système. La performance globale dépend principalement de la qualité de la représentation intermédiaire fournie aux modèles et de leur capacité complémentaire à interpréter les caractéristiques acoustiques.

2 Choix Technologiques

Les outils retenus ont été sélectionnés selon des critères de robustesse, compatibilité scientifique et reproductibilité expérimentale.

— **Python** : langage principal, largement adopté en science des données et en intelligence artificielle.
— **Librosa** : bibliothèque spécialisée dans le traitement audio, utilisée pour le chargement des fichiers et l'extraction des MFCC.
— **NumPy** : gestion efficace des tableaux multidimensionnels et des opérations matricielles.
— **TensorFlow** / **Keras** : implémentation du modèle convolutionnel, gestion de l'entraînement, des métriques et de la sauvegarde des modèles.

L'ensemble de ces technologies assure une cohérence entre implémentation pratique et standards actuels du Deep Learning appliqué à l'audio.

3 Architecture Logicielle

Afin de garantir la modularité du projet, l'organisation logicielle suit une séparation fonctionnelle claire :

```
speech_commands_project/
 data/        # données brutes et transformées
 notebooks/   # expérimentation et analyse
 src/         # code source modulaire
 results/     # modèles et visualisations
 interface/   # inférence en conditions réelles
```

Le dossier `src/` regroupe les composants principaux :
— gestion des données et prétraitement,
— extraction des caractéristiques,
— définition des modèles CNN et LSTM,
— scripts d'entraînement et d'évaluation.

Cette organisation favorise la réutilisabilité du code, la reproductibilité des expériences et l'extension future du système (augmentation de données, modification d'architecture, ajout de nouvelles classes).

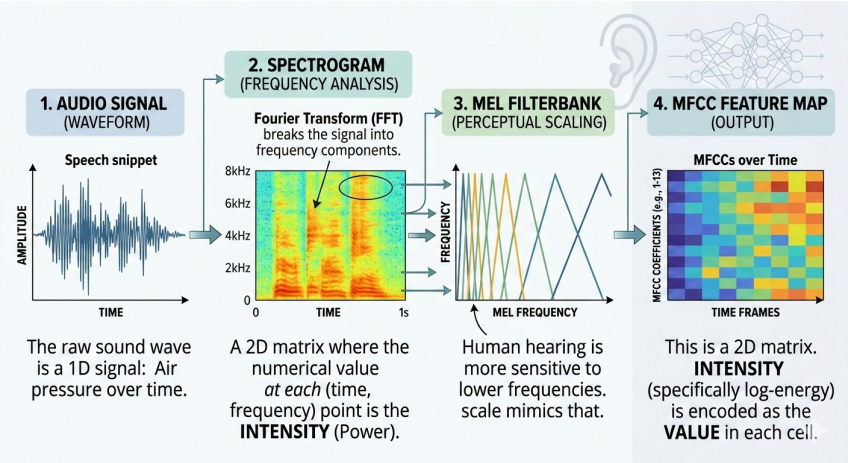
Ainsi, l'architecture globale adoptée constitue un pipeline cohérent, reliant acquisition du signal, transformation spectrale et apprentissage profond au sein d'un environnement logiciel structuré et évolutif.

Prétraitement des Données

4 Chaîne de Prétraitement et Extraction de Caractéristiques

Le signal audio brut constitue une série temporelle échantillonnée à haute fréquence (16 kHz), présentant une forte redondance et une variabilité liée aux conditions d'enregistrement. Sous cette forme, il n'est pas directement exploitable par un modèle de Deep Learning. Une étape de transformation est donc nécessaire afin d'obtenir une représentation compacte, robuste et homogène.

Le pipeline de prétraitement adopté comprend quatre opérations successives.



1. Rééchantillonnage

Tous les enregistrements sont ramenés à une fréquence d'échantillonnage uniforme de 16 kHz. Cette étape garantit l'homogénéité du dataset et assure une bande fréquentielle suffisante pour capturer les composantes pertinentes de la parole, tout en maîtrisant le coût computationnel.

2. Extraction des MFCC

Afin de convertir le signal temporel en représentation structurée, nous extrayons les *Mel-Frequency Cepstral Coefficients* (MFCC).

Les MFCC constituent une représentation temps-fréquence compacte, fondée sur une pondération perceptuelle des fréquences (échelle de Mel) suivie d'une compression spectrale par Transformée en Cosinus Discrète (DCT).

Cette transformation permet :
— de réduire la dimension du signal brut,
— de conserver l'enveloppe spectrale pertinente pour la reconnaissance,
— d'obtenir une structure matricielle adaptée à un CNN 2D et à LSTM comme une séquence de 44 pas de temps, chacun contenant 40 caractéristiques. .

Dans notre implémentation, chaque trame est représentée par 40 coefficients MFCC.

3. Normalisation globale

Pour limiter l'influence des variations d'amplitude entre enregistrements et assurer la cohérence des valeurs sur l'ensemble du dataset, une normalisation globale est appliquée.

La moyenne μ et l'écart-type σ sont calculés uniquement sur l'ensemble du jeu d'entraînement. Les matrices MFCC des ensembles d'entraînement, de validation et de test sont ensuite standardisées selon :

$$MFCC_{norm} = \frac{MFCC - \mu}{\sigma + \epsilon}$$

où ϵ est un terme de stabilité numérique pour éviter la division par zéro.

Cette approche :
— préserve les relations relatives entre fichiers et classes,
— facilite la convergence du modèle,
— améliore la stabilité de l'apprentissage et la performance sur l'ensemble des données.

4. Homogénéisation dimensionnelle

Les réseaux convolutionnels nécessitent des entrées de dimension fixe. Les matrices MFCC sont donc ajustées à une longueur temporelle cible de $T = 44$ trames :
— troncature si la séquence est trop longue,
— zero-padding si elle est trop courte.

La représentation finale est une matrice de dimension $(40, 44)$, à laquelle est ajoutée une dimension de canal, conduisant au tenseur d'entrée :

$$(40, 44, 1)$$

Ainsi, le signal audio brut est transformé en une représentation spectrale normalisée globalement, compacte et compatible avec l'architecture convolutionnelle retenue. Cette étape constitue le pivot entre traitement du signal et apprentissage profond, et conditionne directement la performance du modèle de classification.

Conception du Modèle CNN

5 Justification du choix CNN 2D

Le choix d'un réseau convolutionnel 2D (CNN 2D) repose sur la nature des données traitées dans ce projet : les matrices MFCC extraites des signaux audio. Ces matrices sont des représentations temps-fréquence où chaque dimension (temps et fréquence) contient des motifs locaux pertinents pour la classification des commandes vocales.

Les CNN 2D présentent plusieurs avantages pour ce type de données :

- **Extraction automatique de caractéristiques :** Les filtres convolutionnels apprennent à détecter des motifs discriminants dans les matrices MFCC, sans nécessiter de feature engineering manuel.
- **Invariance locale :** Grâce aux convolutions et au pooling, le réseau est capable de reconnaître des motifs similaires même s'ils apparaissent à différentes positions dans le temps ou dans la fréquence.
- **Efficacité computationnelle :** Les CNN 2D exploitent la structure spatiale des données tout en limitant le nombre de paramètres par rapport à un réseau fully connected.
- **Expérience éprouvée :** Les CNN 2D sont largement utilisés pour la reconnaissance vocale et l'analyse de spectrogrammes, ce qui garantit une robustesse et une bonne performance pour ce type de problème.

Ainsi, l'utilisation d'un CNN 2D constitue un choix pertinent pour traiter des représentations temps-fréquence et classifier efficacement des commandes vocales.

6 Architecture détaillée

L'architecture du réseau de neurones convolutionnel adoptée dans ce projet est composée de plusieurs blocs convolutionnels permettant d'extraire progressivement des caractéristiques pertinentes à partir des matrices MFCC.

- **Bloc 1 :**
  - Couche Conv2D avec 32 filtres de taille 3×3, padding "same"
  - Fonction d'activation ReLU
  - Batch Normalization pour stabiliser l'apprentissage
  - MaxPooling 2×2 pour réduire la dimension spatiale
- **Bloc 2 :**
  - Couche Conv2D avec 64 filtres
  - Batch Normalization
  - MaxPooling 2×2
- **Bloc 3 :**
  - Couche Conv2D avec 128 filtres

  - Batch Normalization
  - MaxPooling 2×2
- **Partie classification :**
  - Flatten : transformation en vecteur 1D
  - Dense (128 neurones, ReLU)
  - Dropout (0.3) pour limiter l'overfitting
  - Dense final avec activation Softmax pour la classification multi-classes

Cette architecture permet d'extraire des motifs locaux dans les représentations MFCC tout en réduisant progressivement la dimension des données.

7 Fonction de perte

La fonction de perte utilisée est la **cross-entropy catégorielle sparse** (*sparse categorical cross-entropy*).

Elle est particulièrement adaptée aux problèmes de classification multi-classes lorsque les étiquettes sont représentées sous forme d'entiers.

Cette fonction mesure l'écart entre la distribution de probabilité prédite par le modèle et la distribution réelle des classes, et guide ainsi l'apprentissage en minimisant cette erreur.

8 Stratégie d'entraînement

L'entraînement du modèle est réalisé selon les paramètres suivants :

- **Optimiseur :** Adam, choisi pour sa rapidité de convergence et sa robustesse
- **Batch size :** 32, permettant un bon compromis entre stabilité et performance
- **Nombre d'epochs :** 30, suffisant pour permettre au modèle d'apprendre les caractéristiques du signal
- **Validation :** utilisation d'un jeu de validation pour surveiller la performance du modèle

Avant l'entraînement, les données sont normalisées à l'aide de la moyenne et de l'écart-type calculés sur le jeu d'entraînement, ce qui améliore la stabilité numérique et la convergence du modèle.

Résultats et analyse du modele CNN

9 Accuracy finale

L'évaluation du modèle est réalisée à la fin de l'entraînement sur les ensembles d'entraînement et de validation.

Les performances obtenues sont les suivantes :

— **Accuracy sur l'entraînement :** 93.85%
— **Loss sur l'entraînement :** 0.2066
— **Accuracy sur la validation :** 88.88%
— **Loss sur la validation :** 0.4600

Ces résultats montrent que le modèle atteint une bonne performance sur les données d'entraînement tout en conservant une capacité de généralisation satisfaisante sur les données de validation.

La proximité entre l'exactitude (accuracy) d'entraînement et de validation témoigne d'une bonne capacité de généralisation du modèle, ce qui est tout à fait satisfaisant pour une tâche de classification audio. Ces scores élevés confirment que le système identifie avec précision la grande majorité des commandes vocales.

10 Matrice de confusion

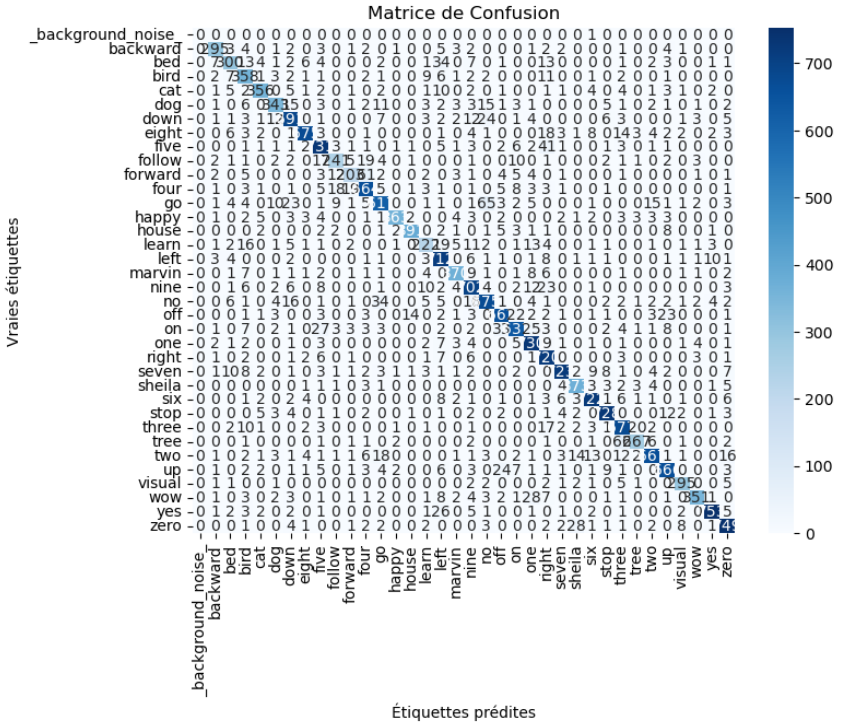


FIGURE 1 – Matrice de confusion du modèle CNN sur le jeu de test. Les valeurs sur la diagonale représentent les classifications correctes, tandis que les valeurs hors diagonale correspondent aux erreurs. Cette matrice met en évidence les confusions entre certaines classes, souvent dues à des similarités phonétiques entre les commandes vocales.

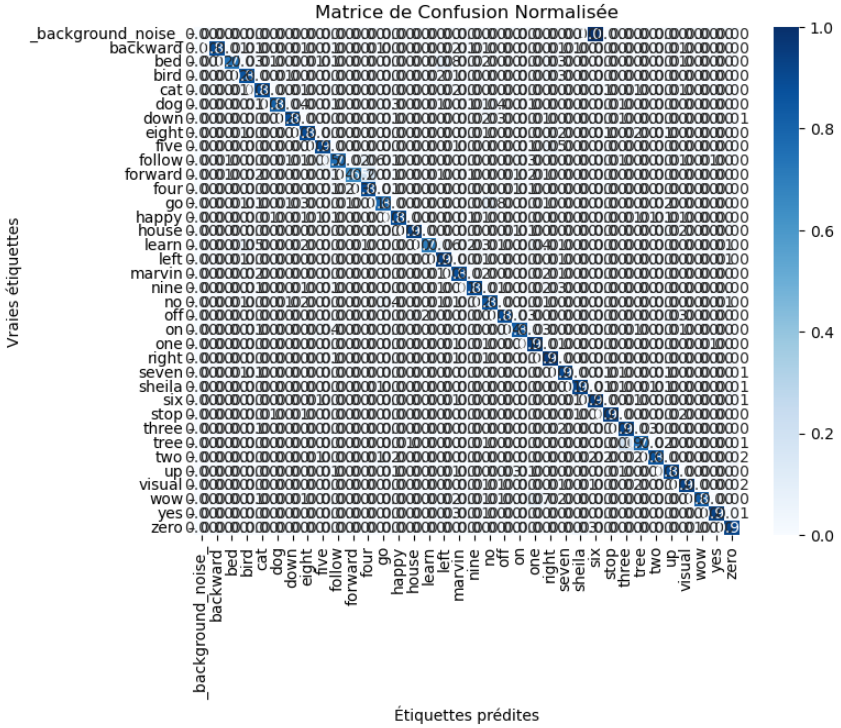


FIGURE 2 – Matrice de confusion normalisée du modèle CNN sur le jeu de test. Les valeurs sont exprimées en proportion par rapport au nombre total d'exemples par classe. Cette représentation permet de visualiser plus facilement les confusions entre classes même si certaines classes sont plus nombreuses que d'autres.

11 Analyse ROC

La courbe ROC a été tracée pour chacune des classes. Les valeurs d'AUC obtenues sont proches de 1 pour toutes les classes, ce qui démontre une capacité élevée du modèle à distinguer correctement les différentes commandes vocales :

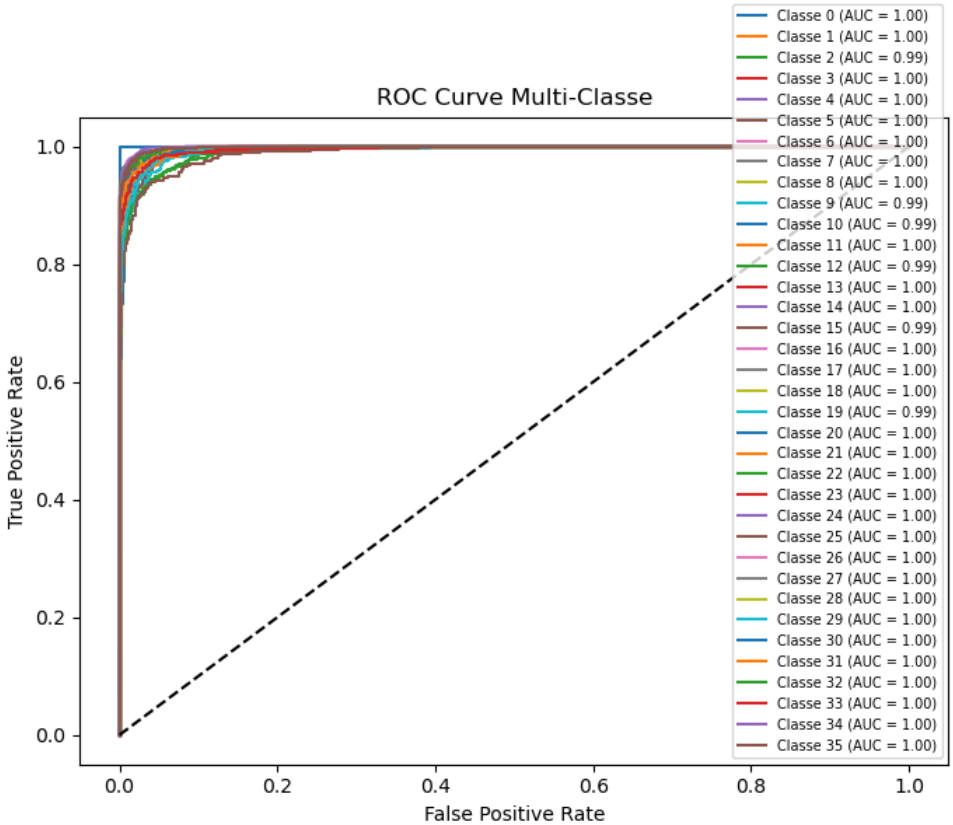


FIGURE 3 – Courbes ROC multi-classes pour le modèle CNN.

12 Analyse de l'Overfitting et Underfitting

Pour évaluer l'équilibre entre apprentissage et généralisation, nous comparons les performances sur les ensembles d'entraînement et de validation :

- **Train Accuracy :** 93.85%, **Train Loss :** 0.2066
- **Validation Accuracy :** 88.88%, **Validation Loss :** 0.4600
- L'accuracy élevée sur l'ensemble d'entraînement montre que le modèle a bien appris les caractéristiques discriminantes des commandes vocales, ce qui exclut un sous-apprentissage (underfitting).
- L'écart modéré entre l'accuracy d'entraînement et de validation indique un léger surapprentissage (overfitting), mais celui-ci reste limité et acceptable.
- Les techniques de régularisation employées (Dropout, Batch Normalization et normalisation des données) contribuent à limiter le surapprentissage.
- Les courbes ROC multi-classes, avec des AUC proches de 1 pour toutes les classes, confirment une excellente capacité du modèle à généraliser correctement sur l'ensemble de validation.

Conclusion : Le modèle présente un bon compromis entre performance et généralisation, sans signe significatif d'underfitting ni d'overfitting excessif.

13 Limites du modèle

Malgré ses bonnes performances, le modèle présente certaines limitations :

- Sensibilité au bruit et aux conditions d'enregistrement
- Confusion entre certaines classes proches
- Dépendance à la qualité des MFCC extraits
- Absence de prise en compte complète des dépendances temporelles (limitation du CNN)

Ces limitations peuvent affecter la précision dans des conditions réelles.

14 Comparaison avec autres approches

Le modèle CNN utilisé présente plusieurs avantages :

- Extraction automatique des caractéristiques, contrairement aux méthodes classiques (SVM, KNN)
- Bonne performance sur les données spectrales (MFCC)
- Complexité modérée et temps d'entraînement raisonnable

Cependant, d'autres approches peuvent être envisagées :

- Les modèles LSTM ou RNN pour capturer les dépendances temporelles
- Les architectures hybrides (CNN + LSTM)
- Les réseaux profonds comme ResNet ou CRNN

Ainsi, le CNN constitue un bon compromis entre simplicité, performance et coût computationnel.

Approche Séquentielle : Modèle LSTM

15 Conception de l'Architecture LSTM

Parallèlement à l'approche convolutionnelle, nous avons implémenté un réseau **LSTM** (Long Short-Term Memory). L'objectif est de traiter les 44 trames temporelles des MFCC comme une séquence chronologique, permettant ainsi au modèle de capturer la dynamique temporelle de la parole.

L'architecture se compose de trois blocs récurrents superposés (32, 64 et 128 unités), chacun suivi d'une couche de *Batch Normalization* et d'un *Dropout* de 0.3 pour maximiser la robustesse du modèle.

16 Résultats Expérimentaux

L'évaluation du modèle LSTM sur le jeu de test a produit les métriques suivantes :

— **Courbes d'apprentissage :** Les graphiques de perte et de précision montrent une progression stable, bien que plus exigeante en ressources calculatoires que le CNN.

— **Matrice de Confusion :** Comme pour le CNN, les erreurs se concentrent sur les commandes phonétiquement proches, bien que le LSTM semble mieux distinguer les variations de durée des mots.

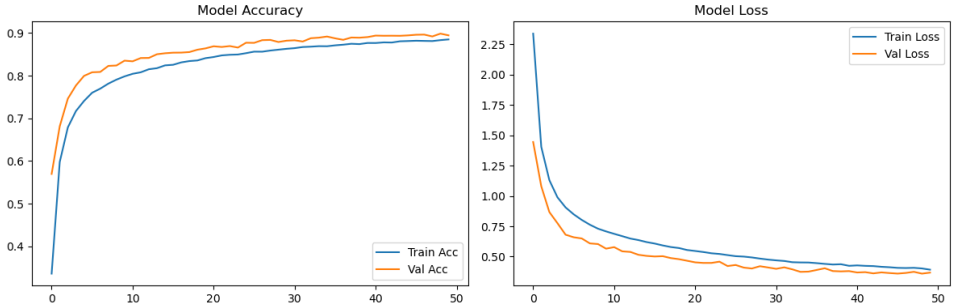


FIGURE 4 – Historique d'entraînement du modèle LSTM.

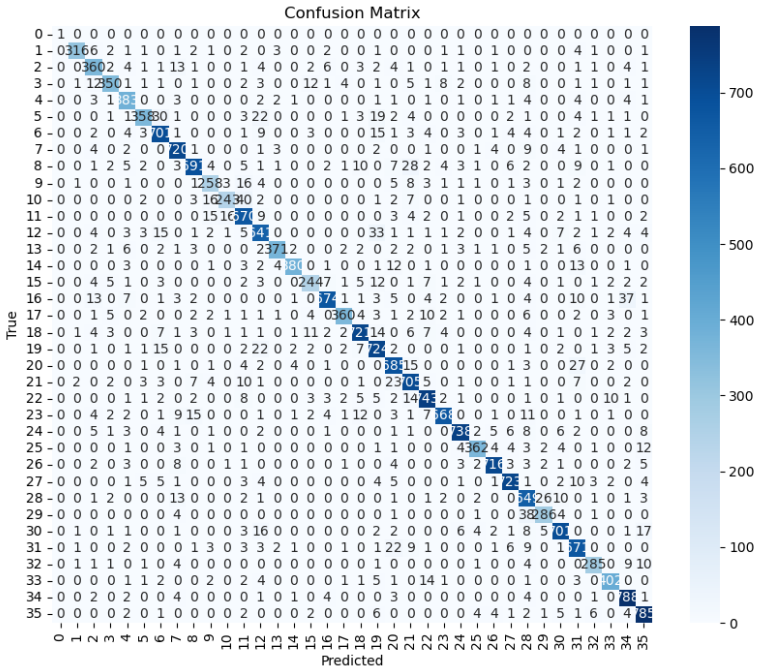


FIGURE 5 – Matrice de confusion du modèle LSTM.

17 Synthèse Comparative

L'intégration du LSTM dans notre pipeline permet d'offrir une alternative robuste au CNN. Alors que le CNN excelle dans l'identification de signatures spectrales fixes, le LSTM apporte une meilleure compréhension de l'évolution du signal dans le temps. Dans notre architecture backend, ces deux modèles peuvent être utilisés de concert pour valider les prédictions.

Architecture Backend et Inférence

Afin de rendre le modèle exploitable en conditions réelles, une API backend a été développée à l'aide du framework Flask. Cette API permet de recevoir un fichier audio, de le traiter et de retourner une prédiction en temps réel.

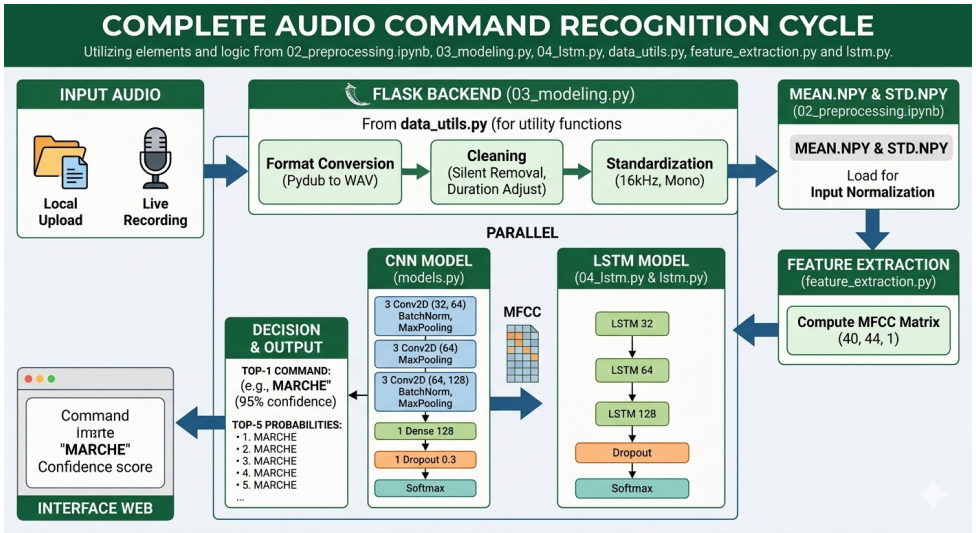


FIGURE 6 – Cycle de vie d'un audio capturé

18 Pipeline d'inférence

Le processus d'inférence suit plusieurs étapes successives :

1. **Réception du fichier audio**
   Le backend reçoit un fichier audio via une requête HTTP POST sur l'endpoint `/predict`.
2. **Conversion audio**
   Le fichier est converti au format WAV, avec un seul canal (mono) et une fréquence d'échantillonnage de 16 kHz afin d'assurer la compatibilité avec le modèle.
3. **Nettoyage du signal**
   Les silences sont supprimés et le signal est ajusté à une durée fixe (1 seconde) par troncature ou padding.
4. **Extraction des caractéristiques**
   Les coefficients MFCC sont extraits avec 40 coefficients et une longueur fixe de 44 frames.

5. **Normalisation**
   Les données sont normalisées à l'aide des paramètres (moyenne et écart-type) calculés lors de l'entraînement.

19 Prédiction

Deux modèles sont utilisés pour la prédiction :
— Un modèle CNN pour l'analyse spatiale des MFCC
— Un modèle LSTM pour capturer les dépendances temporelles

Chaque modèle produit une distribution de probabilités sur les classes, permettant d'identifier la commande la plus probable.

20 Format de sortie

Le backend retourne une réponse au format JSON contenant :
— La commande prédite
— Le niveau de confiance
— Les prédictions alternatives

21 Technologies utilisées

— Flask : création de l'API REST
— TensorFlow / Keras : chargement et exécution des modèles
— Librosa : traitement du signal audio
— Pydub : conversion des fichiers audio
— NumPy : manipulation des données

22 Avantages

Cette architecture présente plusieurs avantages :
— Modularité (séparation frontend / backend)
— Réutilisabilité du modèle
— Facilité de déploiement
— Temps de réponse rapide

Démonstration et Inférence

Pour valider l'efficacité opérationnelle de notre pipeline, nous avons mis en place une interface de démonstration permettant de tester les modèles CNN et LSTM en conditions réelles. Cette étape simule le cycle de vie complet d'une commande vocale : depuis la capture audio jusqu'à l'affichage du résultat. L'utilisateur soumet un échantillon sonore qui est instantanément traité par notre API Flask. Le système exécute alors les deux architectures en parallèle, permettant de comparer leur réactivité et leur précision respective.

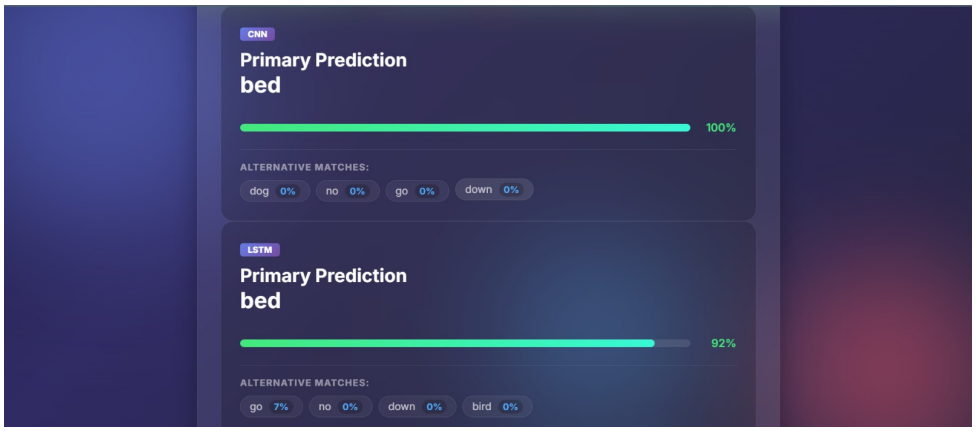


FIGURE 7 – Interface de démonstration : Inférence simultanée utilisant les modèles CNN et LSTM.

Conclusion

Ce projet nous a permis de concevoir et d'implémenter un système complet de reconnaissance de commandes vocales, articulant harmonieusement le traitement du signal et l'apprentissage profond. En transformant des signaux acoustiques bruts en coefficients MFCC, nous avons pu fournir à nos modèles CNN et LSTM une représentation spectrale à la fois compacte et hautement discriminante. Les résultats expérimentaux, marqués par une accuracy dépassant les 88% sur le jeu de validation, démontrent la pertinence des choix architecturaux effectués.

Bien que le modèle CNN se soit révélé particulièrement performant pour l'extraction de motifs fréquentiels, l'approche LSTM a apporté une complémentarité nécessaire pour la gestion de la dynamique temporelle. Ce travail ouvre des perspectives intéressantes, notamment l'exploration de modèles hybrides (CRNN) ou l'intégration de techniques d'augmentation de données pour renforcer la robustesse face aux bruits ambiants. En définitive, cette étude souligne qu'au-delà de la complexité des algorithmes, la qualité du prétraitement et la structuration modulaire du pipeline demeurent les clés de voûte de tout système d'intelligence artificielle appliquée.